

AffineGraph: A Dataflow Graph Representation for Effective DNN Hardware-Aware Algorithms

Chengxiang Qi

University of Chinese Academy of Sciences, Hangzhou Institute for Advanced Study
kuangjux@outlook.com

Unpublished Master’s Research Manuscript – Revised September 2026

Abstract

Hardware-aware algorithms such as FlashAttention and tiled fused operators achieve high GPU performance through careful orchestration of data movement across memory hierarchies. However, existing compiler intermediate representations—whether DAG-based dataflow graphs, polyhedral models, or tile-level abstractions—struggle to simultaneously express the hierarchical memory access patterns and hardware execution constraints that these algorithms exploit. This paper introduces AffineGraph, a compiler framework built on a multi-level dataflow graph IR in which edges carry affine maps that explicitly encode data movement between memory hierarchy levels (global, shared, register). By embedding memory access patterns and loop structure directly into the graph representation, AffineGraph reduces the compiler’s optimization task from searching for legal schedules to resolving hardware-specific parameters—such as tile sizes, shared memory swizzle modes, and instruction selection—within a programmer-specified dataflow structure.

Evaluations on NVIDIA A100 GPUs demonstrate competitive performance across tile-based deep learning workloads. For primitive memory operations, AffineGraph achieves up to 33.3% speedup over CUTLASS. In GEMM, it achieves an average of $0.92\times$ relative to CUTLASS and $1.14\times$ over Triton. For fused two-GEMM operations, where cross-operator data reuse is critical, AffineGraph achieves speedups of up to $3.02\times$ over CUTLASS and $3.26\times$ over Triton by automatically retaining intermediate results in shared memory. In FlashAttention, AffineGraph achieves up to $3.72\times$ speedup over FlashAttention-2 at short sequence lengths, with the advantage narrowing at longer sequences where FlashAttention-2’s hand-tuned tiling becomes more effective. We further demonstrate the portability of our IR design on NVIDIA Hopper (H800), where AffineGraph achieves $\sim 89\%$ of peak HBM3 bandwidth on memory-bound operations (Softmax, RMSNorm), including configurations where Triton-based alternatives fail due to tile size limitations.

1 Introduction

The rapid advancement of deep learning, exemplified by models like Transformers [29] and state-space models (SSM) [13], has driven the development of hardware-aware algorithms that achieve high performance through careful orchestration of GPU memory hierarchies. Algorithms like FlashAttention-2 [12] demonstrate that significant speedups are possible by coordinating data movement across global memory, shared memory, and registers—but implementing such algorithms requires deep expertise in GPU microarchitecture and low-level programming.

Existing compiler frameworks struggle with this class of workloads. DAG-based compilers [6, 8, 33] abstract away memory hierarchy details, making it difficult to express coordinated data movement. Polyhedral compilers [3, 16, 32] provide powerful loop transformation capabilities but face combinatorial search costs when targeting multi-level GPU memory hierarchies, and lack native abstractions for GPU-specific concerns such as shared memory bank conflicts and warp-level instruction selection. Tile-based compilers like Triton [28] simplify kernel authoring but offer limited control over intra-tile data movement and cross-operator data reuse.

We present AffineGraph, a compiler framework that addresses this challenge through a hierarchical dataflow graph IR in which edges carry affine maps encoding data movement between memory hierarchy levels. Unlike search-based approaches that start from a hardware-agnostic representation and explore the space of legal schedules, AffineGraph adopts an *encoding-based* approach: the programmer specifies the algorithmic structure and memory hierarchy mapping in the IR, and the compiler automatically resolves hardware-specific parameters—tile sizes, shared memory swizzle modes, instruction selection, and pipeline scheduling—within the specified structure. This design reduces the compiler’s core task from schedule search (which involves ILP solving in polyhedral frameworks) to parameter optimization within a fixed dataflow topology, a fundamen-

tally simpler problem.

Our work makes the following contributions:

- **A Hardware-Aware Dataflow IR.** We introduce the AffineGraph IR, a hierarchical dataflow graph representation where edges carry affine maps that explicitly encode data movement between memory hierarchy levels (global, shared, register), and nodes represent computation units aligned with GPU execution granularities (block, warp, thread). This design makes memory access patterns and data dependencies directly analyzable from the graph structure, without requiring additional polyhedral analysis passes.
- **Hardware-Conscious Optimization Pipeline.** We develop a three-phase optimization pipeline—graph construction, hardware-conscious transformation, and microtask-based code generation—that leverages the explicit memory hierarchy information in the IR to determine tile sizes, resolve shared memory layouts, select hardware instructions, and schedule asynchronous data movement. The pipeline operates by analyzing the affine maps on graph edges rather than performing exhaustive search over the schedule space.
- **Experimental Evaluation.** We evaluate AffineGraph on NVIDIA A100 across tile-based deep learning workloads including GEMM, fused two-GEMM, FlashAttention, and Block Sparse Attention. AffineGraph achieves competitive performance with CUTLASS on standard GEMM ($0.92\times$ average) while delivering substantial speedups on fused operations (up to $3.02\times$ over CUTLASS) and FlashAttention (up to $3.72\times$ over FlashAttention-2 at short sequences). We additionally demonstrate the portability of our IR design on NVIDIA Hopper (H800), achieving $\sim 89\%$ of peak HBM3 bandwidth on memory-bound operations.

The rest of this paper is organized as follows. Section 2 discusses the limitations of existing compilation approaches and compares AffineGraph with other dataflow representations. Section 3 presents the design of AffineGraph, including its IR, optimization techniques, and a detailed example of progressive lowering from high-level algorithms to hardware-specific code. Section 5 describes our implementation, including how we handle conditional execution patterns and determine tile sizes. Section 6 evaluates the effectiveness of our approach across various matrix shapes and computation patterns. Section 8 concludes.

2 Background and Motivation

The rapid advancement of deep learning has led to increasingly sophisticated hardware-aware algorithms—such

as FlashAttention [12] and tiled fused operators—that require fine-grained control over GPU memory hierarchies and nested parallelism. Implementing these algorithms efficiently demands coordinated data movement across global memory, shared memory, and registers, combined with precise mapping to hardware execution units like Tensor Cores. This trend has exposed fundamental limitations in existing compilation approaches.

Limitations of Existing Approaches. Current approaches to hardware-aware compilation can be broadly categorized into three groups, each with distinct trade-offs:

- **Traditional Dataflow and Deep Learning Compilers.** Existing frameworks like DFGR [25], CnC [20], and deep learning compilers [1, 2, 6, 8, 17, 33] use DAG-based systems or dataflow graphs for expressing dependencies. While effective for coarse-grained parallelization and operator scheduling, they abstract away hardware details such as memory hierarchy levels and warp-level execution, preventing efficient implementation of hardware-specific optimizations like register reuse and coordinated data movement.
- **Polyhedral Frameworks.** Tools like MLIR’s affine dialect [16] and polyhedral compilers such as Tiramisu [3] and AKG [32] provide powerful abstractions for loop transformations and memory access analysis. Their core strength lies in *schedule search*: given a set of affine constraints, they explore the space of legal loop transformations to find performant schedules, typically by solving integer linear programs (ILP) [14]. However, this search-based approach faces two challenges for GPU kernel generation: (1) the search space grows combinatorially with the number of memory hierarchy levels and parallelism dimensions on modern GPUs, and (2) GPU-specific concerns such as shared memory bank conflicts, warp-level instruction selection, and asynchronous memory pipelines lie outside the polyhedral model’s native abstractions, requiring additional ad-hoc passes that are not guided by the polyhedral analysis.
- **Hardware-Specific Compilers and Libraries.** Frameworks like CUTLASS [22] and Triton [28] generate efficient code for specific hardware. CUTLASS provides C++ templates that expose fine-grained hardware control but require expert knowledge to compose. Triton introduces a tile-based programming model that simplifies kernel authoring, but its block-level abstraction limits control over intra-tile data movement—for example, shared memory swizzle strategies and cross-operator register reuse are determined by the compiler with limited user guidance. Both adopt a kernel-centric approach that complicates cross-kernel optimizations

such as fusing consecutive matrix multiplications while retaining intermediate results in shared memory.

Recent efforts to bridge these gaps include AMOS [34] and Roller [36], which introduce hardware features into compilation flows, and Welder [26] and Chimera [35], which employ constrained optimization for kernel fusion. However, these approaches still rely on complex analysis phases to *recover* hardware mapping information that was abstracted away by their IRs, rather than directly expressing hardware patterns in the representation itself.

The Encoding–Search Trade-off. The approaches above reflect a fundamental design trade-off in compiler IR design for GPU kernels:

- **Search-based approaches** (polyhedral compilers, auto-tuners) start from a hardware-agnostic representation and *search* for efficient hardware mappings. This provides generality but incurs high compilation cost and may miss GPU-specific optimization opportunities that fall outside the search model.
- **Encoding-based approaches** (CUTLASS templates, hand-written CUDA) *directly encode* hardware-specific data movement and execution patterns. This yields high performance but sacrifices programmability and portability.

AffineGraph occupies a specific point in this trade-off space: it uses an encoding-based IR—where the programmer specifies the algorithmic structure and memory hierarchy mapping—combined with a compiler that automatically resolves hardware-specific details (tile sizes, swizzle patterns, instruction selection, pipeline scheduling) within the encoded structure. This design reduces the compiler’s task from *searching for a legal schedule* to *optimizing within a specified dataflow*, which is a fundamentally simpler problem.

Distinction from Polyhedral Models. Since AffineGraph uses affine maps as its core formalism, it is important to clarify its relationship to polyhedral compilation. The two approaches share mathematical notation but address different problems:

- In polyhedral compilers, affine maps define the *original* access patterns of a program, and the compiler’s task is to find an affine *schedule* (a new execution order) that satisfies all data dependencies while optimizing for locality or parallelism. This schedule search involves solving ILP problems whose complexity grows with the number of statements and loop dimensions [14].
- In AffineGraph, affine maps encode the *target* data movement pattern—how data flows between specific memory hierarchy levels under a given tiling and parallelism strategy. The compiler does not search for a

schedule; instead, it verifies the legality of the encoded dataflow (a polynomial-time operation on the graph structure) and then determines hardware-specific parameters such as swizzle modes and instruction selection.

This distinction means AffineGraph is not a replacement for polyhedral compilation in general-purpose loop optimization. Rather, it targets a specific and practically important class of GPU workloads—tile-based operators with hierarchical memory access patterns—where the algorithmic structure is known and the primary challenge is efficient hardware mapping.

Key Insights. Our investigation of hardware-aware algorithms reveals two insights that motivate AffineGraph’s design:

- **Affine-Based Memory Modeling.** For tile-based GPU operators, data movement across memory hierarchy levels (global → shared → register) can be precisely described by affine transformations parameterized by loop indices and thread coordinates. Encoding these transformations directly in the IR makes memory access patterns explicit and analyzable without additional compiler passes.
- **Hierarchical Dataflow Structure.** GPU execution naturally forms a hierarchy (grid → block → warp → thread) that mirrors the memory hierarchy. A dataflow graph that reflects this hierarchy through nested subgraphs enables the compiler to reason about optimizations at each level independently—for example, selecting swizzle patterns at the shared memory level without affecting global memory access decisions.

These insights motivate AffineGraph’s design, which introduces hardware-aware abstractions that capture both algorithmic intent and hardware mapping within a unified graph representation.

3 Design

Based on the analysis in Section 2, we design AffineGraph as a compiler framework that introduces a hierarchical, hardware-aware intermediate representation (IR) and an optimization pipeline for tile-based fusion and code generation. The key design principle is that for tile-based GPU operators, encoding memory hierarchy transitions and data movement patterns directly in the IR—rather than recovering them through compiler analysis—simplifies the optimization pipeline while preserving the information needed for hardware-specific code generation.

AffineGraph translates hardware-aware algorithm descriptions into optimized CUDA code through a three-phase lowering pipeline (Figure 1):

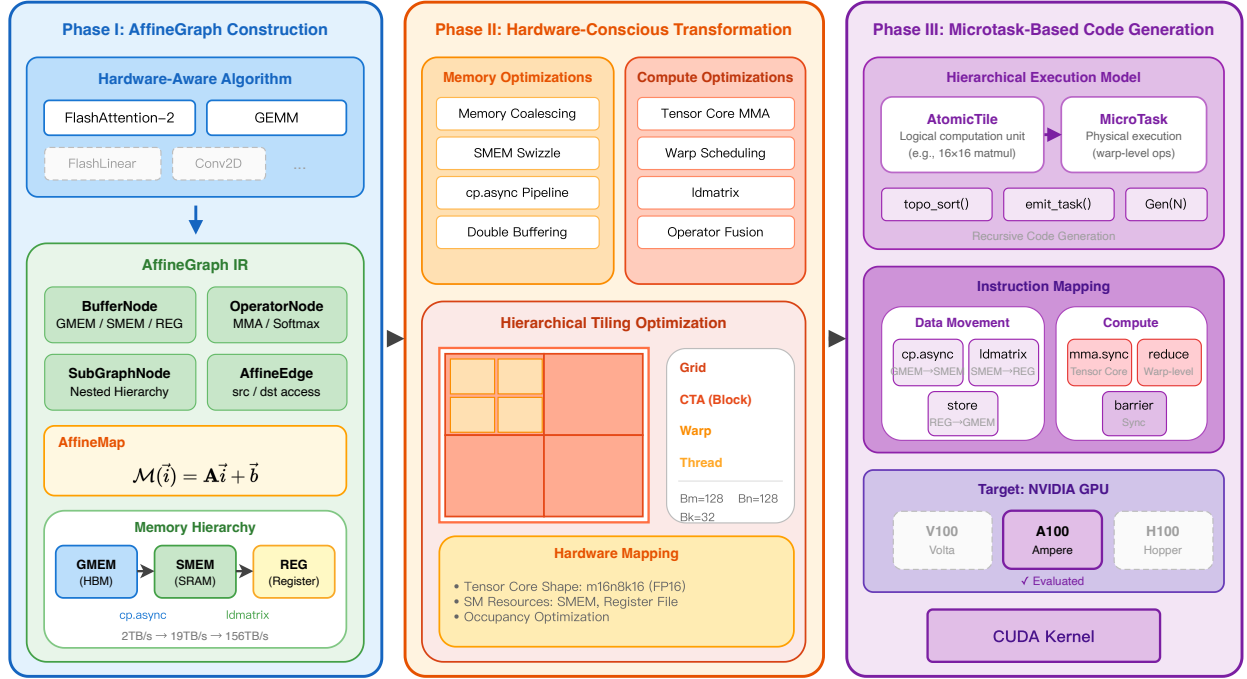


Figure 1: System overview of AffineGraph. The framework consists of three phases: (I) **AffineGraph Construction**: Hardware-aware algorithms such as FlashAttention-2 and GEMM are transformed into AffineGraph IR, which captures memory hierarchy (GMEM $\rightarrow$ SMEM $\rightarrow$ REG) and data access patterns via AffineMap; (II) **Hardware-Conscious Graph Transformation**: Architecture-specific optimizations including memory coalescing, SMEM swizzling, asynchronous memory pipelining (e.g., cp.async, TMA), Tensor Core MMA mapping, and hierarchical tiling; (III) **Microtask-Based Code Generation**: AtomicTiles are mapped to MicroTasks with precise instruction selection, generating optimized CUDA kernels for NVIDIA GPUs.

(I) **Hierarchical AffineGraph Construction**. The programmer specifies the algorithmic structure as a multi-level dataflow graph that explicitly captures memory hierarchy transitions and nested loop structures through affine transformations. This representation enables precise modeling of hardware-aware algorithms like FlashAttention [12], where data movement across memory levels dictates performance.

(II) **Hardware-Conscious Graph Transformation**. The compiler applies architecture-specific optimizations—including tile size selection, shared memory layout inference, and pipeline scheduling—based on GPU microarchitecture parameters. Unlike search-based approaches [6, 33] that explore a large schedule space, AffineGraph’s transformation phase leverages the explicit memory hierarchy information already present in the IR to make targeted optimization decisions within the programmer-specified dataflow structure.

(III) **Microtask-Based Code Generation**. The code generator maps optimized dataflow graph subtrees to warp-level execution units, selecting hardware-specific instructions (e.g., cp.async, ldmatrix, mma.sync) based on the memory hierarchy annotations on graph edges. This approach targets NVIDIA GPU architectures with Tensor Core

support [10, 11].

3.1 AffineGraph IR

The AffineGraph IR serves as the cornerstone of our compiler framework, designed to address the limitations of existing IRs in expressing hardware-aware algorithms. Figure 2 illustrates the three core abstractions of our IR design: **AffineEdge**, **AffineNode**, and **AffineGraph**. At its core, AffineGraph IR introduces a hierarchical representation that combines dataflow graphs with explicit memory access patterns. Nodes are connected by Affine Edges that encode precise memory access patterns through Affine Maps. This design enables direct modeling of hardware-specific features while maintaining high-level algorithmic abstractions. Unlike traditional DAG-based or polyhedral representations that abstract away hardware details, AffineGraph IR provides explicit abstractions for memory hierarchies, nested parallelism, and affine transformations. This enables fine-grained optimizations for modern NVIDIA GPUs by precisely modeling hardware-specific characteristics like memory bank conflicts and specialized compute units.

To demonstrate the expressiveness and optimization capa-

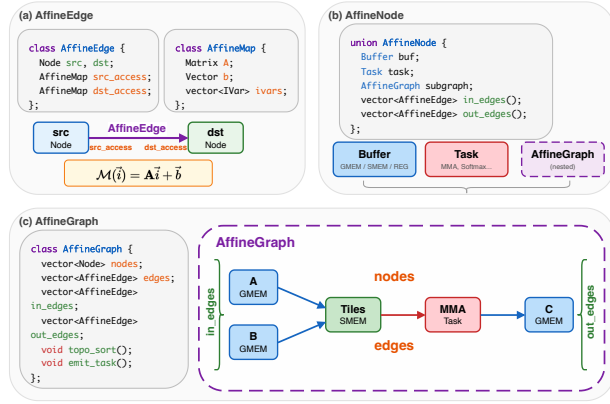


Figure 2: The core abstractions of AffineGraph IR: (a) **AffineEdge** connects nodes via AffineMap-based access patterns; (b) **AffineNode** provides a unified abstraction for Buffer, Task, and nested AffineGraph; (c) **AffineGraph** composes nodes and edges into a hierarchical dataflow graph.

bilities of the AffineGraph IR, we present FlashAttention-2 as a concrete example. Figure 3 illustrates how AffineGraph IR naturally captures the key optimizations of FlashAttention-2 [12], including hierarchical tiling of Q, K, V , memory hierarchy transitions, and nested parallelism. The IR’s structured representation enables automatic optimization of complex memory access patterns and computation dependencies that are typically handled manually in existing frameworks.

Buffer Node. A Buffer Node represents a contiguous memory region at a specific hardware memory hierarchy level. Each buffer is annotated with **Data type**, **Shape**, **Memory space**, and **Alias relationships**. Unlike DAG-based representations that treat memory as opaque tensors, Buffer Nodes encode hardware-specific memory characteristics. For example, shared memory buffers in NVIDIA GPUs enforce bank conflict resolution patterns through swizzled layouts.

Operator Node. An Operator Node describes compute operations acting on Buffer Nodes, representing hardware-aware computation units such as Tensor Core MMA and warp-level reductions. The node is parametrized by instruction-level constraints such as matrix shapes for MMA, multi-level parallelism mappings spanning thread-level, warp-level, and block-level execution granularities, and data reuse opportunities. This granularity enables direct mapping to hardware primitives while maintaining optimization flexibility.

SubGraph Node. SubGraph Nodes represent nested parallel computation units that encapsulate loop nests with affine bounds, tiling strategies with explicit tile shapes and iteration orders, and memory promotion rules that define how buffers move between hierarchy levels. A SubGraph Node recursively contains subgraphs of Buffer, Operator and SubGraph Nodes, enabling hierarchical optimization. For instance, a GEMM SubGraph Node may contain sub-blocks for loading tiles from global to shared memory, loading tiles from shared to register, followed by register-level MMA operations.

Affine Edge. As shown in Figure 2(a), an **AffineEdge** connects different nodes and encodes data dependencies between memory buffers. Each edge carries two **AffineMaps** (`src_access` and `dst_access`) that precisely describe how data is transferred across memory levels. The source and destination nodes of an AffineEdge are annotated with their hierarchy levels, enabling the mapping of edges to distinct hardware operations. For instance, data movement between shared memory and register memory – whether loading or storing – must be mapped to separate hardware operations due to differing synchronization requirements and instruction pipelines.

AffineNode. As shown in Figure 2(b), Buffer, Task (Operator), and SubGraph are encapsulated as a unified **AffineNode** abstraction. Each node maintains `in_edges()` and `out_edges()` interfaces for graph connectivity, enabling uniform traversal and transformation regardless of the underlying node type.

Affine Map. The **Affine Map** is the mathematical core that formally defines memory access patterns. It describes a mapping from an m -dimensional logical iteration space, typically derived from nested loops, to an n -dimensional data coordinate space representing the coordinates of a tensor.

Formally, an Affine Map $\mathcal{M}$ is a function $\mathcal{M} : \mathbb{Z}^m \rightarrow \mathbb{Z}^n$ defined by a tuple $(\mathbf{A}, \vec{b})$ over an iteration domain $\mathcal{D} \subseteq \mathbb{Z}^m$. The mapping is expressed as:

$$\mathcal{M}(\vec{i}) = \mathbf{A}\vec{i} + \vec{b}, \quad \forall \vec{i} \in \mathcal{D}$$

where:

- $\vec{i}$ is an m -dimensional integer vector representing the iteration variables, such as loop indices.
- $\mathbf{A} \in \mathbb{Z}^{n \times m}$ is an integer matrix, representing the linear part of the transformation (strides and permutations).
- $\vec{b} \in \mathbb{Z}^n$ is an integer vector, representing the constant offsets.
- $\mathcal{D}$ is the iteration domain, typically a Z-polyhedron defined by a set of affine inequalities $\mathbf{D}\vec{i} + \vec{d} \geq \vec{0}$.

This structure provides a precise and analyzable representation of complex access patterns like tiling, striding, and interleaving.

To ground this formalism in a practical example, consider a canonical GEMM operation $C_{ij} = \sum_k A_{ik} B_{kj}$ with row-major memory layouts. The iteration domain is $\mathcal{D} = \{(i, j, k)^T \mid 0 \leq i < M, 0 \leq j < N, 0 \leq k < K\}$.

The access to an element $A[i, k]$ in a 2D matrix A of shape $M \times K$ can be described as a map from the iteration space $\mathbb{Z}^2$ (with iterator vector $\vec{i}_A = (i, k)^T$) to a 1D memory address space $\mathbb{Z}^1$. For a row-major layout, the address is calculated as $\text{offset} = i \cdot K + k$. This linear address calculation can be perfectly captured by our affine map definition:

$$\text{addr}(\vec{i}_A) = \begin{bmatrix} K & 1 \end{bmatrix} \begin{bmatrix} i \\ k \end{bmatrix} + [\text{base_addr}_A]$$

Here, the affine map is defined by $\mathbf{A} = \begin{bmatrix} K & 1 \end{bmatrix}$ and $\vec{b} = [\text{base_addr}_A]$. This example demonstrates how the formal matrix-vector notation concisely models concrete, low-level memory access patterns, enabling the compiler to analyze and transform them.

AffineGraph. As shown in Figure 2(c), an **AffineGraph** is a directed graph that represents the computation and dataflow of a hardware-aware algorithm. The graph maintains a collection of nodes and internal edges, along with explicit `in_edges` and `out_edges` for interfacing with parent graphs in hierarchical compositions. Key methods include `topo_sort()` for dependency ordering and `emit_task()` for code generation. This structure enables precise modeling of both the algorithmic logic and hardware-specific optimizations within a unified representation.

Figure 3 demonstrates how **AffineGraph IR** inherently supports FlashAttention-2’s key optimizations through its structured representation. The hierarchical tiling of Q, K, V matrices is naturally expressed via nested **SuperGraph Nodes**, where each block explicitly governs data movement between memory hierarchies. This contrasts with framework-specific tiling implementations by preserving the semantic relationship between tile sizes and their target memory spaces through constraints.

The IR’s affine edge dependencies enable fused computation of matrix multiplies (MMA) with subsequent softmax and dropout operations. By encoding data dependencies between operators as affine maps rather than imperative buffer passing, the compiler can verify fusion legality through static access pattern analysis, reducing the manual validation effort required in CUDA or Triton implementations.

A key differentiation lies in pipeline scheduling: **Affine Edge** explicitly annotates memory-compute overlap opportunities through their source and destination memory hierarchy metadata. This allows the compiler to automatically determine where to insert asynchronous memory operations and

barrier synchronization based on the edge annotations. Compared with raw CUDA or ThunderKittens, where users must manually place memory barriers and manage asynchronous pipelines, AffineGraph’s IR-level annotations shift this burden to the compiler for the data movement patterns expressible in the IR.

Handling Control Flow and Predication

A key challenge for static dataflow representations is handling dynamic control flow, such as that found in masked attention. AffineGraph addresses this through *predication*, where operations are conditionally executed based on a runtime mask. This is achieved by augmenting nodes and edges with an optional *predicate*, rather than introducing explicit control flow nodes that would disrupt the dataflow structure.

We formally define a **Predicate** as a tuple $P = (B_{\text{pred}}, \mathcal{A}_{\text{pred}})$, where:

- B_{pred} is a reference to a **Buffer Node** containing boolean (or integer 0/1) values, representing the condition mask.
- $\mathcal{A}_{\text{pred}}$ is an **Affine Map** that maps the current iteration variables $\vec{i}$ from an iteration domain $\mathcal{D}$ to an address within B_{pred} .

This predicate P can be attached to both data movement edges and computation nodes:

Predicated Affine Edge. An **Affine Edge**, which defines a data transfer from a source buffer B_{src} to a destination buffer B_{dest} via a map $\mathcal{A}_{\text{data}}$, can be predicated by P . The data transfer for a given iteration $\vec{i} \in \mathcal{D}$ is executed *if and only if* the condition $B_{\text{pred}}[\mathcal{A}_{\text{pred}}(\vec{i})] = \text{true}$ is met.

Predicated Operator Node. Similarly, an **Operator Node** can be predicated by P . The computation defined by the operator for an iteration $\vec{i} \in \mathcal{D}$ is executed *if and only if* $B_{\text{pred}}[\mathcal{A}_{\text{pred}}(\vec{i})] = \text{true}$.

For example, in a masked self-attention mechanism, the matrix multiplication of Query and Key matrices can be implemented as a predicated **Operator Node**. Here, B_{pred} would be the attention mask tensor, such as an upper-triangular matrix of boolean values. For each element (i, j) of the output attention score matrix, the map $\mathcal{A}_{\text{pred}}$ would access the corresponding (i, j) element of the mask. The code generator would then emit code that only performs the multiply-accumulate operation if the retrieved mask value is true.

This approach allows AffineGraph to retain its static, analyzable dataflow structure while incorporating data-dependent control flow for algorithms such as masked attention and block sparse attention.

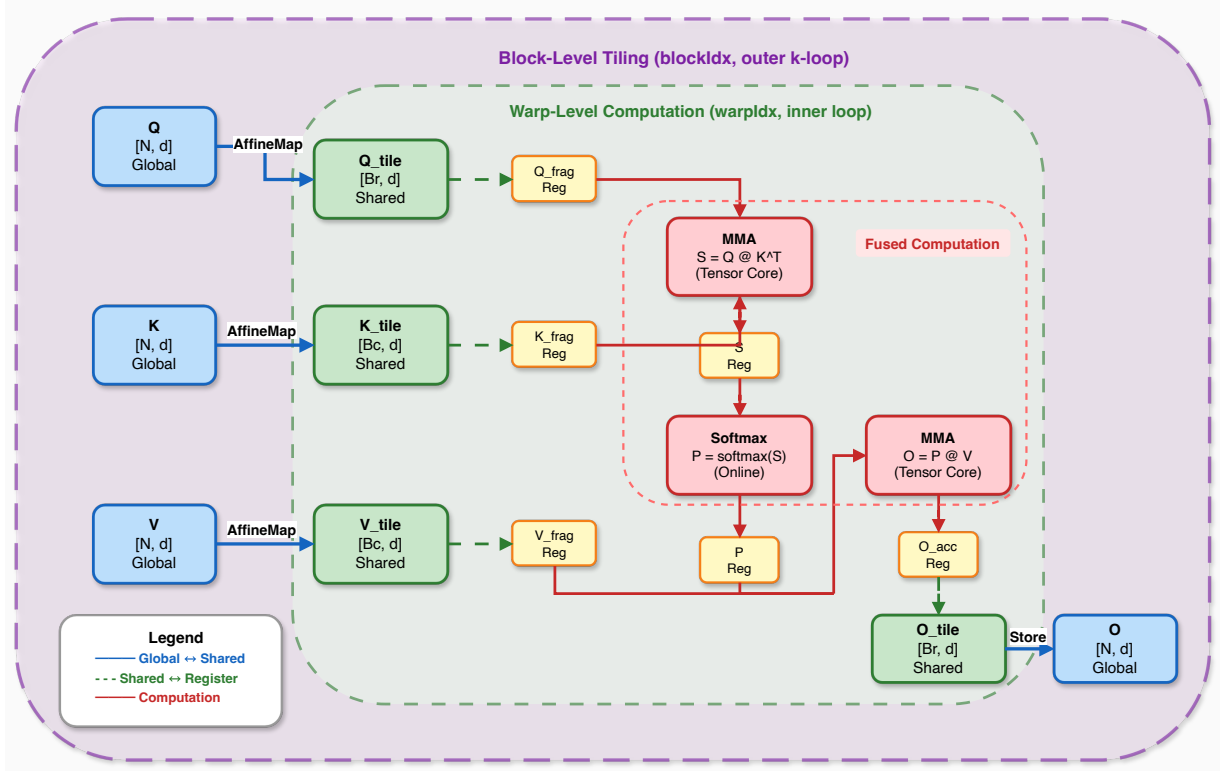


Figure 3: FlashAttention-2 dataflow representation in AffineGraph IR. The figure illustrates nested SuperGraph Nodes capturing hierarchical tiling at block-level and warp-level. Buffer Nodes are explicitly annotated with memory spaces: Global (blue) for Q, K, V, O in HBM; Shared (green) for tiled data in SRAM; and Register (yellow) for computation fragments. Operator Nodes (red) represent Tensor Core MMA operations and online Softmax. Affine Edges encode data movement with AffineMap transformations. The fused computation region shows MMA and Softmax operations executed without intermediate memory writes, with data dependencies converging at the merge point before the final MMA operation.

3.2 Graph Lowering and Code Generation

The Graph Lowering phase translates the optimized Affine-Graph IR into executable code. The process systematically converts the declarative graph representation into an imperative sequence of nested loops and hardware-specific instructions. This is achieved by performing a topological sort of the graph’s nodes and invoking the recursive generation function `GEN`, described in Algorithm 1, on each node in the sorted order.

The recursive `GEN` procedure is the core of the code generation process. It dispatches on the node type (N) to emit the corresponding code. When it encounters a **SubGraph**, it generates the loop header and then recursively calls itself on the subgraph’s children to generate the loop body.

The key logic resides in handling an **Operator** node. The algorithm first iterates through all its incoming data dependencies (edges) and emits the appropriate data loading instructions via an abstract `INSTRUCTIONFORLOAD` function. This function is responsible for generating specific hardware instructions—`cp.async` for GMEM-to-SMEM transfers and

`ldmatrix` for SMEM-to-register loads—based on the memory spaces of the edge’s source and destination buffers. After emitting all necessary loads, it emits a synchronization instruction such as `barrier` to wait for asynchronous operations to complete, before finally emitting the operator’s computation instruction. This formal, recursive definition allows for a clean and systematic translation from the hierarchical graph representation to structured, high-performance code.

3.3 Hardware-Conscious Optimizations

Given the programmer-specified dataflow structure, Affine-Graph’s compiler automatically determines hardware-specific parameters through a *hardware mapping* process. We describe the key decisions below.

3.3.1 Determining Tile Sizes and Parallelism

A key transformation is *tiling*. Given the programmer-specified dataflow structure, the compiler must determine concrete tile sizes at each memory hierarchy level and the

Algorithm 1 AffineGraph Code Generation Algorithm

```
1: procedure GEN(N)
2:   if  $N$  is predicated by  $P = (B_{\text{pred}}, \mathcal{A}_{\text{pred}})$  then
3:     Emit("if ( $B_{\text{pred}}[\mathcal{A}_{\text{pred}}(\vec{i})]$ ) {")
4:   end if
5:   if  $N$  is a SubGraph with domain  $\mathcal{D}_N$  then
6:     Emit("for  $\vec{i} \in \mathcal{D}_N$  {")
7:     for all child  $C$  in TopologicalSort( $N$ .children) do
8:       GEN( $C$ )
9:     end for
10:    Emit("}")
11:   else if  $N$  is an Operator then
12:     for all edge  $E$  in  $N$ .incoming_edges do
13:       Emit(InstructionForLoad( $E$ ))
14:     end for
15:     Emit(InstructionForSync( $N$ ))
16:     Emit(InstructionForCompute( $N$ ))
17:   end if
18:   if  $N$  is predicated then
19:     Emit("}")
20:   end if
21: end procedure
```

parallelism granularity (thread block and warp configuration). We formalize this as a constrained optimization problem.

Configuration Space. A *tiling configuration* $\mathcal{T}$ is a tuple $(T_{\text{blk}}, T_{\text{warp}}, W, S)$ where $T_{\text{blk}} = (M_b, N_b, K_b)$ is the thread block tile size, $T_{\text{warp}} = (M_w, N_w, K_w)$ is the warp tile size, $W = (W_r, W_c)$ is the warp layout within a block, and S is the number of pipeline stages for asynchronous memory operations.

Hardware Constraints. A configuration $\mathcal{T}$ is *feasible* if it satisfies the following constraints derived from GPU microarchitecture parameters:

1. **Tensor Core alignment.** Warp tile dimensions must be integer multiples of the native MMA instruction shape. For FP16 on A100 (`mma.sync.m16n8k16`): $M_w \bmod 16 = 0, N_w \bmod 8 = 0, K_w \bmod 16 = 0$.
2. **Warp decomposition.** The block tile must decompose evenly into warp tiles: $M_b = W_r \cdot M_w$ and $N_b = W_c \cdot N_w$, with the total number of warps $W_r \cdot W_c \leq 32$ (maximum warps per block).
3. **Shared memory capacity.** The total shared memory required for input tiles and pipeline buffers must not exceed the per-SM limit:

$$S \cdot (\text{sizeof}(A_{\text{tile}}) + \text{sizeof}(B_{\text{tile}})) \leq \text{SMEM}_{\text{max}}$$

where $\text{sizeof}(A_{\text{tile}}) = M_b \cdot K_b \cdot \text{sizeof}(\text{element})$ and $\text{SMEM}_{\text{max}} = 164 \text{ KB}$ on A100.

4. **Register pressure.** The register usage per thread, estimated as the accumulator fragment size plus operand fragments, must allow at least N_{min} concurrent blocks per SM:

$$\frac{M_w \cdot N_w}{\text{warpSize}} \cdot \text{sizeof}(\text{acc}) + R_{\text{operand}} \leq \frac{R_{\text{max}}}{N_{\text{min}} \cdot W_r \cdot W_c \cdot \text{warpSize}}$$

where $R_{\text{max}} = 65536$ registers per SM and $N_{\text{min}} \geq 2$.

Selection Heuristic. Among all feasible configurations, the compiler selects the one that maximizes a scoring function:

$$\text{Score}(\mathcal{T}) = \text{Occupancy}(\mathcal{T}) \times \text{ComputeIntensity}(\mathcal{T})$$

where $\text{Occupancy}(\mathcal{T})$ is the ratio of active warps to maximum warps per SM (determined by shared memory and register usage), and $\text{ComputeIntensity}(\mathcal{T}) = (M_b \cdot N_b \cdot K_b) / (M_b \cdot K_b + N_b \cdot K_b)$ captures the ratio of compute operations to data loaded from global memory. This product balances two competing objectives: larger tiles increase data reuse (higher compute intensity) but reduce occupancy due to greater resource consumption.

Because the AffineGraph IR already fixes the dataflow structure and memory hierarchy mapping, the configuration space is small—typically tens to hundreds of feasible configurations—making exhaustive evaluation of the scoring function practical without iterative profiling. The framework can be extended with profile-guided auto-tuning for workloads where the heuristic proves insufficient.

3.3.2 Hierarchical Execution Model

To effectively map high-level operations to the GPU’s parallel architecture, AffineGraph employs a hierarchical execution model built upon two core concepts: **AtomicTile** and **Micro-Task**.

AtomicTile. We define an **AtomicTile** as the fundamental *logical* unit of computation, representing the smallest matrix operation, such as a 16×16 matrix multiplication, that the compiler schedules. It specifies **what** computation needs to be done.

Micro-Task. In contrast, a **Micro-Task** is the *physical* execution sequence that implements an AtomicTile. It defines **how** a warp executes the logical computation, orchestrating a series of hardware-level operations, including data movement across memory hierarchies and core computations, as shown in Figure 4.

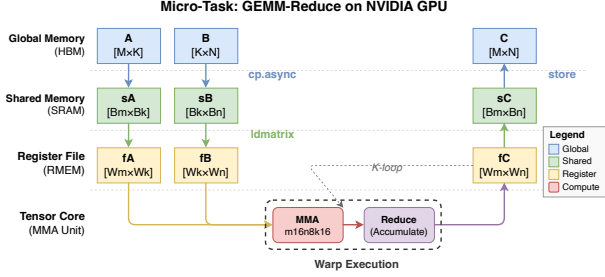


Figure 4: Micro-Task execution model for GEMM-Reduce on NVIDIA Ampere GPU. The figure illustrates the physical execution sequence of a warp implementing an AtomicTile, showing data movement across memory hierarchies (Global $\rightarrow$ Shared $\rightarrow$ Register) via `cp.async` and `ldmatrix` instructions, followed by Tensor Core MMA computation and accumulation. The K-loop indicates iterative reduction across the K dimension.

By separating the logical computation unit (AtomicTile) from its physical execution plan (Micro-Task), AffineGraph can systematically manage the complexity of code generation while ensuring efficient hardware utilization.

3.3.3 Effective Memory Access Optimization

A key component of generating efficient Micro-Tasks is optimizing memory access.

Global Memory Access. AffineGraph optimizes global memory access patterns based on warp configurations. A key hardware constraint is that a single warp’s memory transaction cannot exceed 1024 bits, which corresponds to the L1 cache line size. AffineGraph leverages this knowledge to select the optimal warp organization for a given tensor shape, minimizing bandwidth waste.

Shared Memory Swizzling. To mitigate shared memory bank conflicts, AffineGraph applies *swizzling* techniques to the shared memory layout, inspired by libraries like CUTLASS [22]. The compiler uses just-in-time (JIT) compilation to evaluate different strategies, such as Swizzle(2,3,3) versus Swizzle(3,3,3), and select the optimal approach that balances bank conflict avoidance with memory pressure for the specific workload.

3.3.4 Adaptation to Hopper Architecture

AffineGraph’s core abstractions—AffineMap, hierarchical memory modeling, and MicroTask—are parameterized by memory hierarchy levels and instruction types rather than tied to a specific GPU generation. We describe how these abstractions map to three key NVIDIA Hopper (SM90) [21]

architecture features, and evaluate the resulting performance in §6.3:

(1) *Tensor Memory Accelerator (TMA)*. Hopper introduces hardware-accelerated asynchronous bulk data movement between global and shared memory via the Tensor Memory Accelerator (TMA). In AffineGraph’s IR, TMA operations are modeled as specialized AffineEdges for GMEM $\rightarrow$ SMEM transfers, where the data movement is offloaded to a dedicated hardware engine rather than requiring per-thread address calculation. The AffineMap encoding of access patterns remains unchanged; only the INSTRUCTIONFORLOAD function in the code generation phase (Algorithm 1) selects TMA descriptors and `cp.async.bulk` instructions instead of per-thread `cp.async`.

(2) *Thread Block Clusters*. Hopper introduces a new level in the execution hierarchy—clusters of cooperating thread blocks that can directly access each other’s shared memory. AffineGraph’s hierarchical SubGraph Node abstraction accommodates this by introducing an additional nesting level above the thread block, extending the existing Thread $\rightarrow$ Warp $\rightarrow$ Block hierarchy to Thread $\rightarrow$ Warp $\rightarrow$ Block $\rightarrow$ Cluster. This extension enables cluster-level cooperative data loading and cross-block shared memory access without fundamental changes to the IR design.

(3) *Warpgroup-Level Operations*. Hopper supports warp-group (128-thread) collective operations, including `wgmma` instructions for Tensor Core operations. In AffineGraph, these are expressed as Operator Nodes with larger AtomicTile shapes, where the MicroTask execution plan orchestrates warpgroup-level computation instead of individual warp-level MMA. The tile size determination heuristics (§3.1) are extended to account for warpgroup granularity and Hopper’s increased register file and shared memory capacity.

4 Case Study

4.1 Hierarchical IR Construction and Dataflow Representation

Matrix multiplication (GEMM) is a cornerstone of modern deep learning and scientific computing workloads. Achieving high performance for GEMM on GPUs requires a sophisticated strategy for managing data movement across the memory hierarchy, from global memory to shared memory and registers. Compilers typically rely on vendor-tuned, handwritten libraries such as cuBLAS or complex heuristics to generate efficient kernels. The former are often opaque and hard to customize, while the latter struggle to consistently find optimal configurations across different hardware and problem sizes.

We use GEMM to demonstrate how *AffineGraph* can declaratively represent a complex kernel, enabling the automatic discovery of high-performance implementations. Fig-

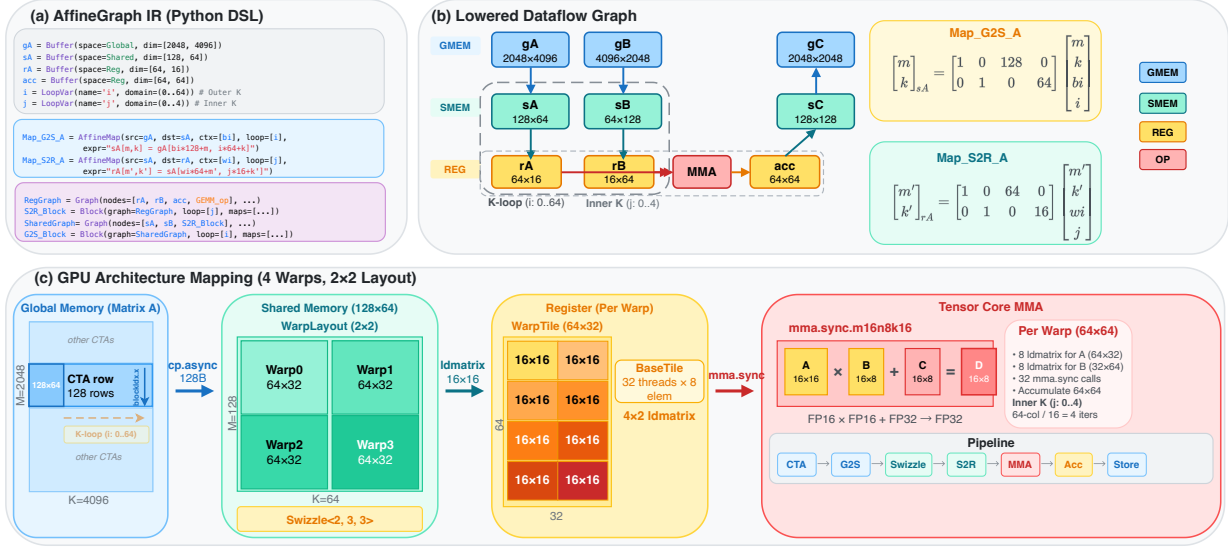


Figure 5: GEMM case study using AffineGraph. (a) AffineGraph IR in Python DSL showing buffer declarations across memory hierarchy (Global, Shared, Register), loop variables for K-dimension tiling, and affine maps defining data movement patterns. (b) Lowered dataflow graph with explicit dependencies and affine transformation matrices annotating each data transfer edge. (c) GPU architecture mapping showing matrix A's data flow: CTA-level tiling with K-loop iteration in global memory, 4-warp (2x2) collaboration with Swizzle<2,3,3> in shared memory, WarpTile (64x32) decomposed into 4x2 ldmatrix tiles in registers, and Tensor Core MMA execution pipeline.

Figure 5 illustrates the process of generating a GEMM kernel using our framework.

Figure 5(a) shows the high-level *AffineGraph* IR for a tiled GEMM implementation. The program begins by declaring *Buffers* for the input and output matrices in their respective memory spaces: *gA* in global memory, *sA* in shared memory, and *acc* for accumulators in registers. The core logic of data tiling and movement is captured by *AffineMap* definitions. For instance, *Map_G2S_A* specifies the transfer of a 128x64 tile from global memory to shared memory, parameterized by the block index (*bi*) and the outer k-loop index (*i*). Similarly, *Map_S2R_A* loads a 64x16 micro-tile from shared memory into a warp's registers. These individual affine mappings are composed into nested *Blocks* and *Graphs*, which directly mirror the nested loop structure of the tiled GEMM algorithm.

This declarative, high-level IR is then lowered into a dataflow graph that makes all dependencies explicit, as shown in Figure 5(b). This graph represents the complete flow of data, from initial loads at the *gA* and *gB* buffers down to the final accumulation in registers. Specifically, the edges representing data transfers, such as the one from *gA* to *sA*, are annotated with the *AffineMap* defined in the IR. This makes the exact data tiling and access patterns an integral part of the dataflow graph, rendering it amenable to a variety of automated analyses and transformations. For example, the *Map_G2S_A* annotation uses a single affine transformation matrix to formally define the data access pattern. This matrix maps the block index (*bi*), loop index (*i*), and local tile

coordinates (*m*, *k*) to the source address in *gA*. This concise mathematical representation is key for enabling the compiler to analyze and optimize complex data movement, rendering the graph amenable to a variety of automated analyses and transformations.

Figure 5(c) provides a detailed view of how the data for matrix A is tiled and mapped to the GPU's execution and memory hierarchy. In global memory, each CTA is assigned a row of the matrix via *blockIdx*, while the K-dimension is traversed through the outer k-loop (*i*: 0..64), loading 128x64 tiles per iteration. In shared memory, four warps in a 2x2 layout collaborate to process a 128x64 tile, with each warp handling a 64x32 region. The Swizzle<2,3,3> transformation is applied to eliminate bank conflicts. Each warp's 64x32 WarpTile is decomposed into 4x2 ldmatrix tiles (16x16 each) for loading into registers. Finally, the *mma.sync.m16n8k16* instruction executes Tensor Core computation, with each warp performing 32 MMA calls to accumulate a 64x64 result. *AffineGraph* provides a unified, mathematical representation for this entire hierarchy of tiling and thread mapping.

4.2 Layout Inference and Transformation

Once the hierarchical IR is constructed, the compiler performs a sophisticated **Layout Inference** process that transforms the logical representation into a concrete, hardware-optimized execution plan. As illustrated in Figure 6, this process consists of three key phases that systematically address

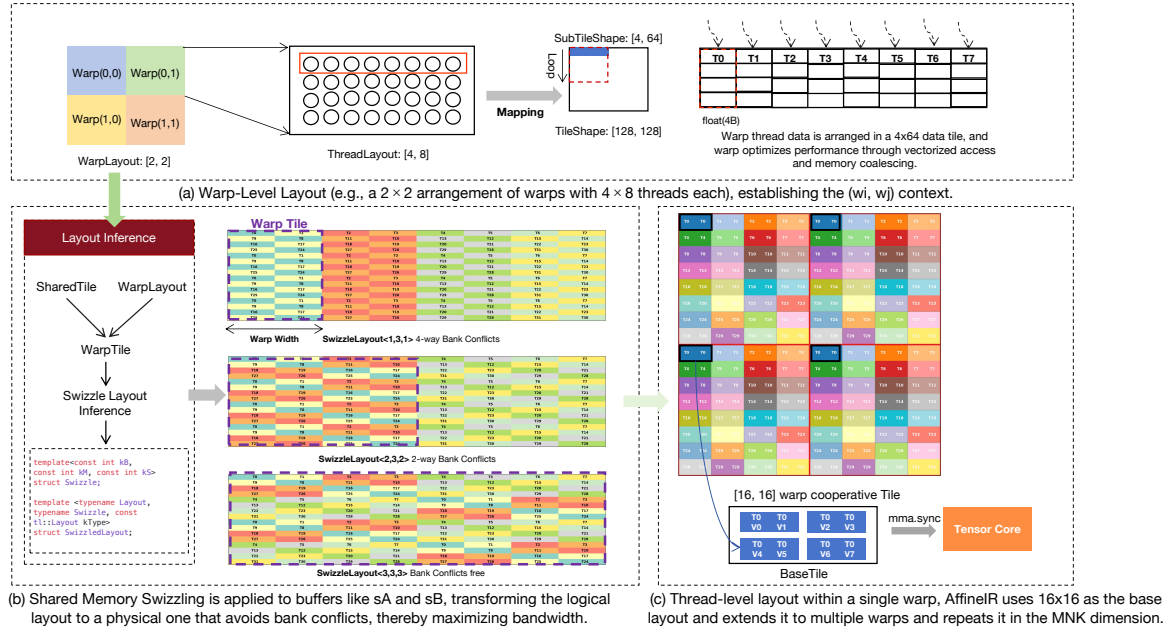


Figure 6: Layout Inference and Transformation pipeline in AffineGraph. (a) Global-to-Shared data loading with automatic vectorization and coalesced memory access based on WarpLayout iteration over GlobalTile. (b) Shared Memory Layout Inference showing how WarpTile configurations are derived from SharedTile and WarpLayout, with adaptive swizzling strategies (Swizzle<B,M,S>) for different tile sizes to balance bank conflicts and CTA occupancy. (c) Register-level BaseTile abstraction where individual threads handle 8 elements, with 32 threads collaboratively processing 16×16 ldmatrix tiles for efficient mma.sync operations.

data movement from global memory to registers, optimizing for both memory bandwidth and computational efficiency.

Global-to-Shared Memory Loading with Automatic Vectorization Figure 6(a) demonstrates how AffineGraph handles the critical first stage of data movement. Given a specific **WarpLayout** configuration, the compiler determines how warps should iterate over the **GlobalTile** to load data into shared memory. This process combines warp-level parallelism with serial for-loop execution to achieve optimal memory throughput.

The compiler automatically applies **vectorization** and **coalesced memory access** optimizations during this phase. For a given tile size, warps collaborate to load contiguous memory regions, with each warp handling multiple elements per thread to maximize memory bandwidth utilization. The iteration pattern is carefully designed to ensure that memory accesses within each warp are coalesced into wide transactions such as 128-byte cacheline loads, while multiple warps work in parallel to cover the entire GlobalTile efficiently.

Shared Memory Layout Inference and Adaptive Swizzling Figure 6(b) illustrates the sophisticated **Layout Inference** mechanism that operates within shared memory. This process takes the **SharedTile** and **WarpLayout** as in-

puts and automatically derives optimal **WarpTile** configurations for different computational scenarios.

A key innovation is the adaptive swizzling strategy, implemented through parameterized **Swizzle<B,M,S>** transformations that handle different memory access patterns:

- **32-byte swizzle:** Optimized for smaller WarpTiles, resulting in 4-way bank conflicts but requiring fewer resources per warp
- **64-byte swizzle:** Balanced configuration with 2-way bank conflicts, suitable for medium-sized tiles
- **128-byte swizzle:** Eliminates bank conflicts entirely but demands more resources per WarpTile

The compiler automatically selects the appropriate swizzling mode based on the WarpTile dimensions, balancing between memory access efficiency and **CTA occupancy**. Larger swizzle patterns reduce bank conflicts but may limit the number of concurrent CTAs due to increased resource requirements, necessitating careful trade-off analysis during compilation.

Register-Level BaseTile Abstraction and Tensor Core Integration Figure 6(c) presents the final stage where data

is organized for register-level computation. AffineGraph introduces a **BaseTile** abstraction that defines the minimal processing unit for individual threads within a warp.

Each thread operates on a BaseTile containing 8 elements, carefully arranged to align with the GPU’s register file architecture. The 32 threads within a warp collaborate to form a larger **WarpTile** that processes exactly the data required for a 16E16 *ldmatrix* operation. This collaborative arrangement enables efficient loading of matrix fragments directly into registers optimized for Tensor Core consumption.

The BaseTile abstraction supports hierarchical nesting, allowing larger WarpTiles to be constructed through serial execution across rows and columns. This design enables warps to handle variable tile sizes while maintaining the fundamental 16E16 granularity required for efficient *mma.sync* operations. The register data is precisely organized to match the input requirements of Tensor Core instructions, ensuring optimal computational throughput.

This three-stage Layout Inference process transforms the user’s high-level, hardware-aware dataflow specification into a series of precisely orchestrated, hardware-specific micro-operations, automatically handling the complex interplay between memory hierarchy optimization and computational efficiency.

4.3 CUDA Code Generation from Dataflow Graph

After the layout inference and transformation stages, *AffineGraph* proceeds to the final code generation phase, translating the optimized dataflow graph into high-performance CUDA code. This process is facilitated by a sophisticated backend built upon a C++ template library. This library provides a collection of reusable components that encapsulate low-level hardware details and warp-level primitives, enabling a modular and extensible approach to kernel generation.

The core of the backend is a set of template classes that represent GPU memory structures (*GlobalTile*, *SharedTile*, *RegTile*) and data movement operations (*GlobalToSharedLoader*, *SharedToRegLoader*). By traversing the dataflow graph, the code generator instantiates these templates with the specific tile shapes, layouts, and data types determined during the earlier optimization phases. This allows the compiler to systematically construct a kernel tailored to the exact problem dimensions and hardware characteristics. For instance, a data transfer edge in the graph from global to shared memory directly maps to the instantiation and invocation of a *GlobalToSharedLoader* object in the generated code.

The listing below shows the high-level structure of a generated GEMM kernel, omitting many implementation details for clarity. The process starts with a *KeGemmTraits* struct that centralizes all compile-time configurations, such as tile

Listing 1: High-level structure of a generated CUDA GEMM kernel.

```

1 // 1. Traits struct centralizes kernel
  configuration.
2 struct KeGemmTraits {
3     using G2SLoaderA = GlobalToSharedLoader<...>;
4     using S2RLoaderA = SharedToRegLoader<...>;
5 };
6 // 2. Kernel implementation uses types from
  Traits.
7 template <typename Traits>
8 __global__ void gemm(const InType* dA, const
  InType* dB, AccType* dC) {
9     typename Traits::G2SLoaderA g2s_a;
10    typename Traits::S2RLoaderA s2r_a;
11    typename Traits::RegC acc;
12    // 3. Main loop for pipelined data movement
  and computation.
13    for (int k = ...) {
14        // Load tiles from global to shared memory
15        g2s_a(gAs(k), sA);
16        g2s_b(gBs(k), sB);
17        __copy_async();
18        __syncthreads();
19        // Inner loop for computation
20        for (int k_inner = ...) {
21            // Load from shared to registers
22            s2r_a(sAs(k_inner), rA);
23            s2r_b(sBs(k_inner), rB);
24            compute::gemm(rA, rB, acc);
25        }
26    }
27    // 4. Store results back to global memory.
28    r2s_c(acc, sC);
29    __syncthreads();
30    s2g_c(sC, gC);
31 }

```

sizes and data layouts. The *gemm* kernel is then instantiated with these traits. Its core logic consists of a nested loop structure that orchestrates the movement of data from global memory to shared memory and finally to registers, where the computation is performed using warp-level primitives. This demonstrates how our template-driven backend translates the dataflow graph into a concrete, highly-structured CUDA kernel.

5 Implementation

Implementation Organization. The implementation is organized across a compiler core and two GPU backend libraries. The AffineGraph compiler core is implemented in Rust and Python. Its core source tree contains approximately 5.0K lines of code: 4.6K lines of Rust for the IR, graph analysis, lowering, target interfaces, and backend code generation, together with 0.4K lines of Python for the graph-building DSL and runtime adapters. The repository additionally contains executable examples and integration tests for GEMM, fused GEMM, FlashAttention, Softmax, and RMSNorm.

The Ampere backend emits CUDA C++ against **TileFusion**, which provides the tile types, memory-copy primitives, Tensor Core operations, and runtime support used by the generated kernels. The current TileFusion core comprises approximately 5.4K source lines across CUDA, C++, and Python, excluding examples, benchmarks, and tests. The Hopper prototype is implemented with **CuteSnippets**; its HTile abstractions and kernel implementations comprise approximately 1.6K lines of Python, excluding benchmarks, tests, and tutorial examples. The AffineGraph compiler repository is maintained at <https://github.com/KuangjuX/AffineGraph>; the backend repositories above are publicly available.

1

Hopper Architecture Prototype. To validate the portability of AffineGraph’s design principles beyond Ampere, we implement a prototype for NVIDIA Hopper (H800) targeting memory-bound operations. This prototype follows AffineGraph’s core methodology—hierarchical tiling with explicit memory hierarchy mapping—and is implemented using NVIDIA’s CuTe DSL [22], which provides Python-level abstractions for Hopper-specific features such as TMA and Thread Block Clusters. In particular, the prototype adopts the same tile-based dataflow design described in §3.3.4: data movement is organized as a hierarchical Global→Shared→Register pipeline with cluster-level cooperative loading for large tile sizes. The prototype demonstrates that AffineGraph’s design naturally extends to Hopper’s enhanced memory subsystem, requiring only backend-level changes to instruction selection while preserving the same IR-level abstractions. Full integration of the Hopper code generation backend into the compiler is an engineering effort that does not require fundamental changes to the IR or optimization pipeline.

6 Evaluation

6.1 Experimental Setup

Hardware Platform. Our primary experiments were conducted on an NVIDIA A100 GPU with 80GB HBM2 memory, featuring 108 Streaming Multiprocessors (SMs) and a peak FP16 tensor core performance of 312 TFLOPS. We use CUDA version 12.1 on Ubuntu 20.04. To evaluate the portability of AffineGraph’s design across GPU generations, we additionally conduct experiments on an NVIDIA H800 GPU (Hopper architecture) with 80GB HBM3 memory and a peak memory bandwidth of 3.35 TB/s (§6.3).

¹Source-line counts were measured with `cloc` 2.10 on September 30, 2026. Counts exclude blank and comment lines, third-party dependencies, documentation, generated artifacts, examples, benchmarks, and tests. They describe the current repositories rather than the historical snapshot used for every experiment in this paper.

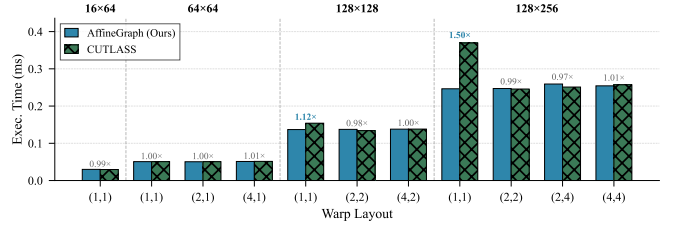


Figure 7: Performance comparison of memory operations (Load/Store) between AffineGraph and CUTLASS across different matrix shapes and warp layouts. Numbers above bars indicate speedup ratios (CUTLASS/AffineGraph). AffineGraph achieves up to 1.50x speedup on larger matrices.

DNN Workloads. Our evaluation focuses on fundamental deep learning operations that form the building blocks of modern neural networks. We evaluate primitive memory operations (Load/Store) with various access patterns to assess basic memory access optimization capabilities. For dense matrix operations, we test standard GEMM with dimensions ranging from 1024x to 8192x, and Batched GEMM with batch sizes from 2 to 16. To demonstrate fusion benefits, we evaluate Fused two GEMMs operations where consecutive matrix multiplications are combined. For attention mechanisms, we test FlashAttention [12] with batch size 32, sequence lengths from 128 to 4096, hidden sizes of 128 and 256, and 8 attention heads. We also evaluate Block Sparse Attention with hidden sizes of 64 and 128, sequence lengths from 128 to 1024, demonstrating our compiler’s ability to handle irregular memory access patterns. These workloads comprehensively cover both memory-bound and compute-bound operations.

Baselines. We compare AffineGraph with state-of-the-art DNN frameworks and libraries. To ensure fair comparison, we describe the configuration of each baseline in detail:

- **CUTLASS [22]:** We implement CUTLASS baselines using NVIDIA’s CuTe DSL primitives (CUTLASS 3.x), which provides fine-grained control over tiled copy, swizzled shared memory layouts, and Tensor Core MMA operations. The CUTLASS kernels use the same `SM80_CP_ASYNC_CACHEGLOBAL` copy atoms and `Swizzle<2,3,3>` shared memory layouts as our implementation, ensuring that performance differences reflect algorithmic and scheduling decisions rather than low-level primitive choices.
- **Triton [28]:** We use Triton’s official tutorial implementations with the `@triton.autotune` decorator enabled, which searches over multiple tile size configurations (varying `BLOCK_SIZE_M`, `BLOCK_SIZE_N`, `BLOCK_SIZE_K`, and `GROUP_SIZE_M`) and selects the best-performing configuration for each input shape.

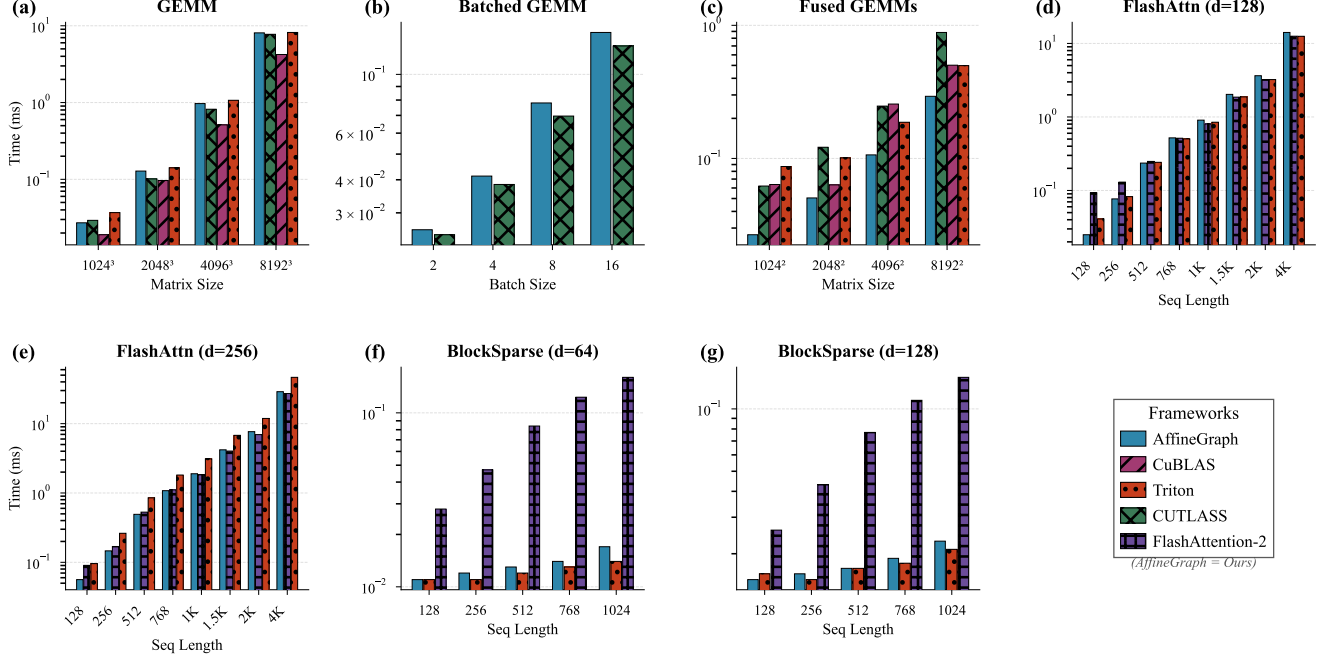


Figure 8: Performance comparison of AffineGraph against baseline frameworks. (a) Standard GEMM across different matrix sizes. (b) Batched GEMM with varying batch sizes. (c) Fused two GEMMs operations. (d) FlashAttention with hidden size 128. (e) FlashAttention with hidden size 256. (f) Block Sparse Attention with hidden size 64. (g) Block Sparse Attention with hidden size 128. Lower execution time is better.

- **cuBLAS** [9]: We use NVIDIA’s cuBLAS library (CUDA 12.1), which internally selects optimized kernel implementations based on matrix dimensions through its heuristic dispatch mechanism.
- **FlashAttention-2** [12]: We use the official FlashAttention-2 implementation, a hand-optimized CUDA kernel with architecture-specific tuning for A100 GPUs.
- **PyTorch** [23]: We use PyTorch’s default operator implementations, which dispatch to cuBLAS for GEMM and use native CUDA kernels for other operations.

For Hopper experiments, we compare against `torch.compile` [2] (PyTorch 2.x’s compiler with default settings) and Liger Kernel [18] (a Triton-based kernel library optimized for LLM training). All evaluations begin with warm-up iterations, followed by repeated execution of each workload for a minimum of 5 seconds to ensure stable measurements. Average performance metrics are calculated using the geometric mean across all configurations within each workload category.

Compilation Overhead. An important practical consideration is the time required to compile a kernel. AffineGraph’s compilation pipeline—from IR construction through graph

transformation to CUDA code generation and `nvcc` compilation—processes each kernel in a single pass without iterative search. We note that AffineGraph’s encoding-based design avoids the iterative compilation required by autotune-based systems: once a kernel is compiled for a given operator, it can be cached and reused across input shapes that share the same tiling structure. In contrast, Triton’s `@triton.autotune` requires re-evaluation of multiple configurations for each distinct input shape.

6.2 Operator Performance on NVIDIA A100

Figure 8 shows the performance of all operator configurations in our evaluation. The x-axis represents different operator configurations, and the y-axis indicates the execution time in milliseconds.

Memory Operations. We first evaluate AffineGraph on primitive Load/Store operations to assess its basic memory access optimization capabilities. Figure 7 shows performance across matrix shapes from $16\text{E}64$ to $128\text{E}256$. AffineGraph achieves performance within 1.3% of CUTLASS on small matrices ($16\text{E}64$ and $64\text{E}64$). As matrix size increases, the performance advantage becomes more pronounced: an 11.2% speedup at $128\text{E}128$ and up to 33.3% speedup at $128\text{E}256$. This improvement stems from Affine-

Graph’s ability to leverage its affine-based analysis to model complex memory access patterns and generate efficient memory hierarchy traversal code. The performance improvements are particularly significant for larger matrices, where memory access patterns become more complex and optimization opportunities are more abundant.

GEMM. Figure 8(a) shows the performance of standard GEMM operations across matrix sizes from 1024s to 8192s. Although existing compilers and libraries already provide well-optimized GEMM kernels, AffineGraph still achieves competitive performance. Compared to CUTLASS, AffineGraph achieves an average performance of 0.92E (up to 1.08E). Against Triton, AffineGraph maintains a consistent advantage with an average speedup of 1.14E (up to 1.36E). While AffineGraph does not consistently outperform highly-tuned vendor libraries like cuBLAS, which leverage years of manual optimization and architecture-specific tuning, achieving competitive performance demonstrates that our compiler-generated code can approach hand-optimized implementations. AffineGraph achieves this performance using its heuristic-based tile size selection and compiler-determined memory access optimizations, without per-shape manual tuning of the generated kernel code. The algorithmic structure (tiled GEMM with K-dimension reduction) is specified by the programmer in the IR.

For Batched GEMM operations (Figure 8(b)), we evaluate configurations with batch sizes ranging from 2 to 16. AffineGraph demonstrates consistent performance across different batch sizes, with execution times within 0.87-0.97E of CUTLASS. The stable performance as batch size increases highlights our framework’s ability to efficiently handle batched operations through effective memory access modeling.

Fused Two GEMMs. Compared to standard GEMM operations, Fused two GEMMs involve more complex computation stages and data dependencies, opening up a larger optimization space for AffineGraph. Figure 8(c) shows the performance of Fused two GEMMs operations across matrix sizes from 1024E1024 to 8192E8192 (with K and P dimensions fixed at 128). AffineGraph demonstrates significant performance advantages across all matrix sizes. Compared to CUTLASS, Triton, and cuBLAS, AffineGraph achieves average speedups of 2.52E (up to 3.02E), 2.18E (up to 3.26E), and 1.94E (up to 2.41E), respectively. This stands in stark contrast to standard GEMM where AffineGraph is competitive but not consistently faster.

This improvement stems from AffineGraph’s ability to express cross-operator data reuse directly in the IR. For the pattern $D = (A \times B) \times C$, the programmer specifies a fused dataflow graph where the intermediate result $A \times B$ resides in shared memory, and the compiler generates a single kernel that avoids the round-trip to global memory. Baseline

approaches either execute separate kernels (cuBLAS, PyTorch), incurring global memory writes/reads between the two GEMMs, or require manual fusion specification with explicit shared memory management (Triton, CUTLASS). This fusion is particularly effective when intermediate dimensions (K, P) are small (128 in our experiments), as the intermediate results fit in shared memory. The consistent 2–3 $\times$ speedups validate a core capability of AffineGraph’s IR: by making memory hierarchy placement explicit in the graph structure, cross-operator optimizations that are difficult to express in kernel-centric frameworks become straightforward to specify and compile.

FlashAttention. FlashAttention involves multiple computation stages (two matrix multiplications interleaved with online softmax) and coordinated data movement across all three memory levels, making it a demanding test for the IR’s ability to express complex dataflow patterns. Figure 8(d-e) shows the performance of FlashAttention operators with hidden sizes of 128 and 256. The evaluations cover sequence lengths ranging from 128 to 4096, with batch size of 32 and 8 attention heads.

AffineGraph demonstrates strong performance particularly at shorter sequence lengths. For hidden size 128 (Figure 8(d)), AffineGraph achieves speedups of 3.72 $\times$ (seqlen=128), 1.69 $\times$ (seqlen=256), and 1.05 $\times$ (seqlen=512) over FlashAttention-2. As sequence length increases, AffineGraph’s advantage diminishes, with execution times closely matching FlashAttention-2 for longer sequences. Compared to Triton, AffineGraph achieves an average speedup of 1.24 $\times$ (up to 3.72 $\times$). For hidden size 256 (Figure 8(e)), AffineGraph shows similar trends with speedups of 1.61 $\times$ (seqlen=128), 1.16 $\times$ (seqlen=256), and 1.08 $\times$ (seqlen=512) over FlashAttention-2, and maintains consistent advantages over Triton with speedups up to 3.06 $\times$.

The sequence-length-dependent performance reflects a trade-off in tile size selection. At shorter sequences, the working set is small enough that AffineGraph’s compiler can select tile configurations that keep most data in shared memory, maximizing reuse. As sequences lengthen, the working set exceeds shared memory capacity, and FlashAttention-2’s hand-tuned tiling strategy—which was specifically optimized for long-sequence regimes—becomes more effective. This pattern suggests that AffineGraph’s heuristic tile selection is effective for moderate problem sizes but does not yet match expert-level tuning for all configurations.

Block Sparse Attention. Figures 8(f-g) present the performance of Block Sparse Attention operations with hidden sizes of 64 and 128, respectively. The configurations we test have sequence lengths ranging from 128 to 1024. Block Sparse Attention introduces irregular memory access patterns and conditional computation, providing a challenging test for compiler optimization.

For hidden size 64 (Figure 8(f)), AffineGraph achieves competitive performance with Triton, with performance ranging from 0.82E to 1.00E across different sequence lengths. Against FlashAttention-2, AffineGraph demonstrates significant advantages with average speedups of 2.5–9.4E. The large gap versus FlashAttention-2 reflects that FlashAttention-2 is optimized for dense attention and incurs overhead when adapted to sparse patterns. Compared to PyTorch, AffineGraph maintains substantial speedups of 15.6–19.0E.

For hidden size 128 (Figure 8(g)), AffineGraph continues to show strong performance. Against Triton, AffineGraph achieves speedups ranging from 0.91E to 1.07E. Compared to FlashAttention-2 and PyTorch, AffineGraph demonstrates average speedups of 1.7–6.2E and 11.9–14.2E, respectively.

These results show that AffineGraph’s predication mechanism (§3.1) can handle the conditional computation and non-contiguous memory access patterns introduced by block sparsity. The competitive performance with Triton—a compiler that also supports masked operations—suggests that AffineGraph’s IR is expressive enough for this class of structured sparse patterns. However, we note that block sparse attention still exhibits regular, tile-aligned sparsity; operators with unstructured sparsity or data-dependent access patterns (e.g., top- k routing in Mixture-of-Experts) would require extensions beyond the current affine map formalism.

6.3 Portability to Hopper Architecture

To evaluate the generality of AffineGraph’s design beyond Ampere, we conduct experiments on NVIDIA H800 GPU (Hopper architecture) with 80GB HBM3 memory and a peak memory bandwidth of 3.35 TB/s. We focus on memory-bound operations—Softmax and RMSNorm—which stress the memory subsystem and benefit significantly from Hopper’s architectural enhancements.

Experimental Setup. We implement Softmax and RMSNorm kernels following AffineGraph’s hierarchical tiling methodology (§3.3.4), leveraging Hopper-specific features including Thread Block Clusters for cooperative data loading and warpgroup-level reductions. We compare against `torch.compile` [2] (PyTorch 2.x’s compiler) and Liger Kernel [18], a Triton-based kernel library optimized for LLM training. All inputs use FP16 precision. Input shapes range from $[M, N] = [32768, 1024]$ to $[4096, 131072]$, covering a wide spectrum of memory access patterns from tall-narrow to short-wide matrices.

Results. Figure 9 presents the results.

For Softmax (Figure 9(a-b)), AffineGraph achieves $1.2\times$ – $2.2\times$ speedup over `torch.compile` across all configurations. Compared to Liger Kernel, AffineGraph maintains comparable performance for moderate sizes ($N \leq 32K$),

while achieving $1.6\times$ speedup at $N = 64K$ and scaling to $N = 128K$ where Liger Kernel fails due to Triton’s tile size limitations. AffineGraph sustains 2765–2996 GB/s of effective memory bandwidth across all shapes, reaching up to 89% of H800’s peak HBM3 bandwidth.

For RMSNorm (Figure 9(c-d)), AffineGraph achieves $1.2\times$ – $1.7\times$ speedup over `torch.compile`. Against Liger Kernel, AffineGraph shows comparable performance at small-to-moderate sizes, but achieves $3.2\times$ speedup at $N = 64K$ where Liger Kernel’s Triton backend becomes inefficient, and scales to $N = 128K$ which Liger Kernel does not support. AffineGraph sustains 2743–2969 GB/s of effective bandwidth throughout.

Analysis. Two key observations emerge from the Hopper results:

(1) *Consistent near-peak bandwidth.* AffineGraph sustains approximately 2900 GB/s ($\sim 87\%$) of H800’s peak bandwidth across all configurations for both Softmax and RMSNorm. This confirms that AffineGraph’s hierarchical tiling methodology—partitioning data across Global→Shared→Register with cluster-level cooperative loading—effectively utilizes Hopper’s enhanced memory subsystem.

(2) *Scalability to large tile sizes.* A key advantage of AffineGraph’s design is its support for large per-row dimensions (N up to 131K) through cluster-level cooperation and multi-stage tiling. Both `torch.compile` and Liger Kernel (Triton-based) either degrade significantly or fail entirely at $N \geq 64K$, because Triton’s tile size is bounded by its compiler’s maximum tile dimensions. AffineGraph’s explicit memory hierarchy modeling enables flexible tiling strategies that partition large dimensions across multiple loading stages within a cluster, avoiding this limitation.

These results provide evidence that AffineGraph’s core abstractions—hierarchical memory modeling and affine-based access pattern analysis—can be adapted to a newer GPU generation for memory-bound workloads. The Hopper prototype requires instruction-level adaptation (TMA instead of `cp.async`, `warpgroup` instead of warp-level MMA) while preserving the same IR-level design, as discussed in §3.3.4. We note that the current Hopper evaluation is limited to memory-bound operators; validating the approach on compute-bound workloads (e.g., GEMM, FlashAttention) on Hopper remains future work that would require full integration of the Hopper code generation backend.

7 Related Work

We discuss related work along four axes: polyhedral and loop-based compilers, tile-based GPU compilers, hardware-specific kernel libraries, and auto-tuning and kernel fusion frameworks.

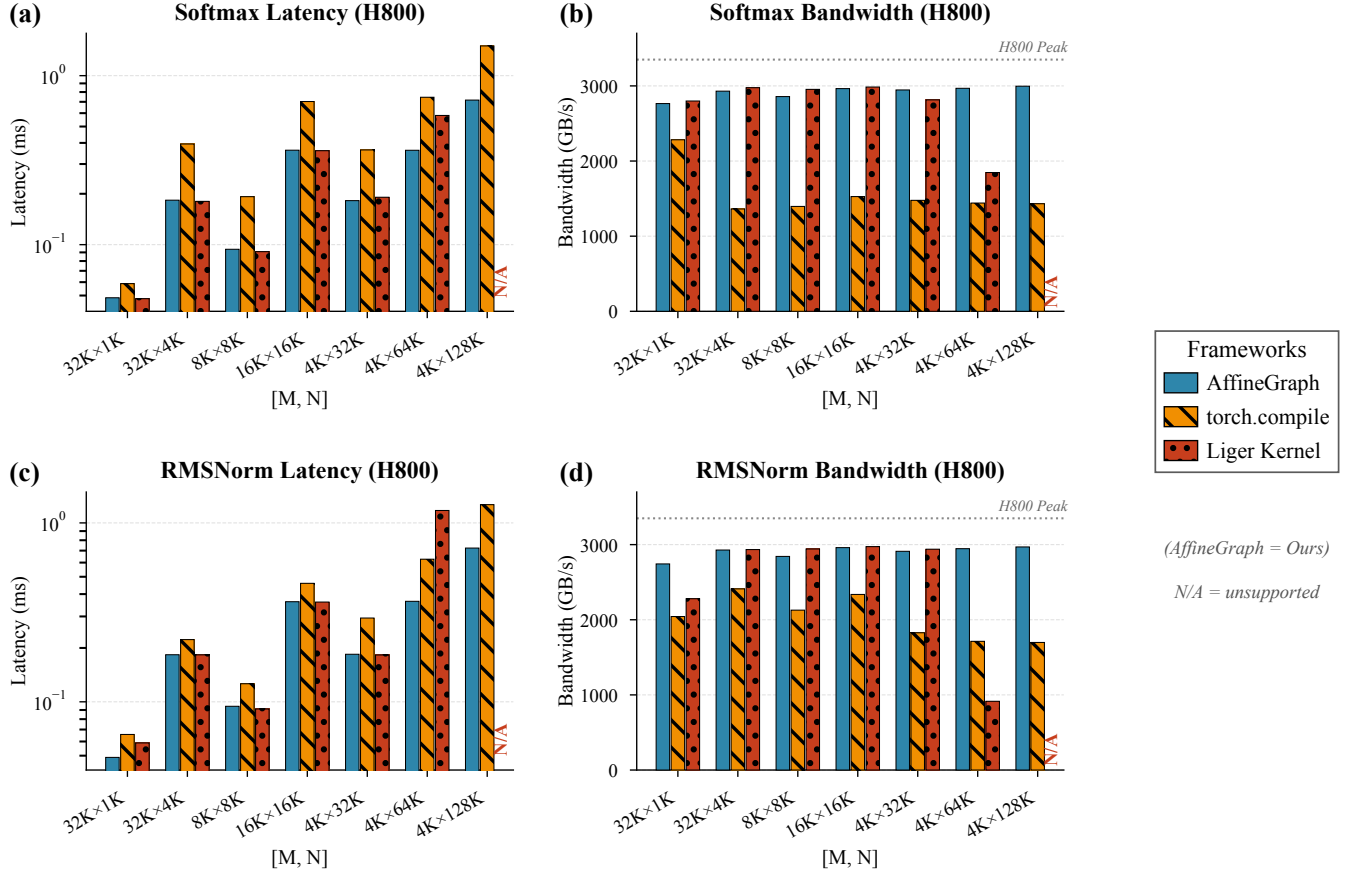


Figure 9: Performance of memory-bound operations on NVIDIA H800 (Hopper). (a) Softmax latency. (b) Softmax effective bandwidth. (c) RMSNorm latency. (d) RMSNorm effective bandwidth. AffineGraph sustains ~ 2900 GB/s across all configurations, approaching H800’s peak HBM3 bandwidth of 3350 GB/s. N/A indicates that the framework does not support the given configuration.

Polyhedral and Loop-Based Compilers. The polyhedral model provides a mathematically rigorous framework for analyzing and transforming loop nests with affine bounds and access patterns [4, 14, 15]. MLIR’s affine dialect [16] brings polyhedral abstractions into a modern compiler infrastructure, enabling affine loop transformations within a multi-level IR. Tiramisu [3] separates algorithm specification from scheduling decisions, allowing users to express loop transformations as scheduling commands applied to a polyhedral representation. AKG [32] applies polyhedral transformations to automatically generate kernels for neural processing units.

AffineGraph shares the use of affine maps with these systems but differs in its design objective. Polyhedral compilers use affine analysis to *search* for legal and profitable loop transformations—a process that involves solving integer linear programs whose complexity grows with the number of statements and loop dimensions. AffineGraph instead uses affine maps to *encode* a target data movement pattern specified by the programmer, reducing the compiler’s task to parameter selection within a fixed dataflow structure. This trade-off sacrifices the generality of automatic schedule dis-

covery in exchange for lower compilation complexity and direct control over GPU-specific optimizations (shared memory swizzling, asynchronous pipelines) that lie outside the polyhedral model’s native abstractions.

Tile-Based GPU Compilers. Triton [28] introduces a tile-based programming model that raises the abstraction level for GPU kernel authoring. Programmers operate on tile-level tensors, and the compiler handles lowering to hardware instructions. This significantly reduces the expertise required to write performant kernels. However, Triton’s block-level abstraction provides limited control over intra-tile data movement: shared memory layout strategies and cross-operator register reuse are determined by the compiler, which can lead to suboptimal decisions for complex operators. AffineGraph’s IR exposes these decisions through explicit affine edge annotations, enabling the programmer to specify—and the compiler to optimize—data movement at a finer granularity.

TileLang [30] proposes a composable tiled programming model that extends tile-level abstractions with ex-

implicit scheduling primitives for pipelining and parallelism. Tawa [5] introduces automatic warp specialization for modern GPUs with asynchronous references. PipeThreader [7] focuses on software-defined pipelining for efficient DNN execution. These systems address complementary aspects of GPU kernel optimization; AffineGraph’s contribution is orthogonal in that it provides a dataflow graph IR where memory hierarchy transitions are first-class citizens, enabling the compiler to reason about cross-operator data reuse and memory placement at the IR level.

Halide [24] pioneered the separation of algorithm from schedule in domain-specific compilation. AffineGraph follows a similar philosophy: the programmer specifies the algorithmic dataflow structure in the IR (analogous to Halide’s algorithm), and the compiler determines hardware-specific parameters (analogous to Halide’s schedule). The key difference is that AffineGraph’s IR is designed specifically for GPU memory hierarchies, with affine maps on graph edges encoding data movement between named memory levels, whereas Halide’s scheduling language targets a broader class of image processing pipelines.

Hardware-Specific Kernel Libraries. CUTLASS [22] provides C++ template abstractions for high-performance linear algebra on NVIDIA GPUs, with fine-grained control over tiling, memory hierarchy, and Tensor Core utilization through its CuTe DSL. ThunderKittens [27] offers a tile-based embedded DSL that leverages Hopper-specific features such as register-level tile abstractions and asynchronous warpgroup operations. FractalTensor [19] uncovers nested data parallelism and data reuse patterns in DNN computations.

These libraries achieve high performance through direct hardware control but require expert-level programming effort. AffineGraph aims to provide a compilation path that approaches the performance of these libraries while offering a higher-level specification interface. Our evaluation shows that AffineGraph matches CUTLASS performance on standard GEMM ($0.92\times$ average) and exceeds it on fused operations (up to $3.02\times$), where AffineGraph’s IR-level cross-operator optimization provides an advantage over CUTLASS’s kernel-centric approach.

Auto-Tuning and Kernel Fusion Frameworks. TVM [6] and Ansor [33] employ search-based auto-tuning to find efficient tensor program implementations. Ansor uses a hierarchical search space with learned cost models to generate high-performance kernels without manual templates. Roller [36] constrains the search space using hardware alignment rules to accelerate compilation. AMOS [34] introduces hardware abstraction for spatial accelerators, enabling automatic mapping of tensor computations.

For kernel fusion, Welder [26] schedules memory access across fused operators using a tile-graph abstraction.

Chimera [35] employs analytical modeling to optimize compute-intensive operator fusion. Ladder [31] enables efficient low-precision computing through hardware-aware tensor transformation.

AffineGraph differs from auto-tuning systems in that it does not search over the schedule space; instead, the programmer encodes the dataflow structure and the compiler selects hardware parameters within that structure. Compared to fusion frameworks, AffineGraph’s hierarchical IR naturally supports cross-operator data reuse by making memory hierarchy placement explicit in the graph, rather than relying on post-hoc fusion analysis. This design choice trades generality (AffineGraph requires programmer involvement in specifying the dataflow) for compilation efficiency and predictable performance.

8 Conclusions

This paper presents AffineGraph, a compiler framework for tile-based GPU operators that encodes memory hierarchy transitions and data movement patterns directly in a hierarchical dataflow graph IR. By shifting the compiler’s task from searching for legal schedules to optimizing hardware-specific parameters within a programmer-specified dataflow structure, AffineGraph achieves competitive or superior performance compared to existing compilers and hand-optimized libraries across a range of workloads.

Our evaluation on NVIDIA A100 demonstrates that AffineGraph’s encoding-based approach is particularly effective for operators with cross-level data reuse opportunities: fused two-GEMM operations achieve up to $3.02\times$ speedup over CUTLASS, and FlashAttention achieves up to $3.72\times$ over FlashAttention-2 at short sequence lengths. For standard GEMM, AffineGraph achieves $0.92\times$ of CUTLASS on average, showing that compiler-generated code from the IR can approach hand-optimized implementations. On NVIDIA Hopper (H800), a prototype targeting memory-bound operations achieves $\sim 89\%$ of peak HBM3 bandwidth, providing initial evidence that the IR abstractions can adapt to newer GPU generations.

Limitations and Future Work. We identify several limitations of the current work that define directions for future research:

- **Scope of evaluated workloads.** Our evaluation focuses on tile-based operators with affine access patterns (GEMM, attention, element-wise fusions). Operators with unstructured sparsity, data-dependent control flow (e.g., top- k routing), or dynamic shapes fall outside the current IR’s expressiveness and are not evaluated.
- **Programmer involvement.** AffineGraph requires the programmer to specify the algorithmic structure and

memory hierarchy mapping in the IR. While the compiler automates hardware-specific parameter selection (tile sizes, swizzle modes, instruction selection), the initial dataflow specification requires understanding of the target algorithm’s tiling strategy. Reducing this specification burden—for example, through higher-level algorithm templates or guided IR construction—is an important direction.

- **Heuristic-based parameter selection.** The current tile size and parallelism selection relies on heuristic models rather than a formal cost model or profile-guided search. While effective for the evaluated workloads, this approach may miss optimal configurations for operators with unusual resource trade-offs. Integrating a lightweight cost model that leverages the IR’s explicit memory hierarchy information is a natural extension.
- **Hopper evaluation scope.** The Hopper prototype currently covers only memory-bound operators (Softmax, RMSNorm). Extending the full code generation backend to support compute-bound workloads on Hopper—including wgmma-based GEMM and FlashAttention—requires additional engineering effort and is planned as future work.

References

- [1] ABADI, M., BARHAM, P., CHEN, J., CHEN, Z., DAVIS, A., DEAN, J., DEVIN, M., GHEMAWAT, S., IRVING, G., ISARD, M., ET AL. {TensorFlow}: a system for {Large-Scale} machine learning. In *12th USENIX symposium on operating systems design and implementation (OSDI 16)* (2016), pp. 265–283.
- [2] ANSEL, J., YANG, E., HE, H., GIMELSHEIN, N., JAIN, A., VOZNESENSKY, M., BAO, B., BELL, P., BERARD, D., BUROVSKI, E., ET AL. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (2024), pp. 929–947.
- [3] BAGHDADI, R., RAY, J., ROMDHANE, M. B., DEL SOZZO, E., AKKAS, A., ZHANG, Y., SURIANA, P., KAMIL, S., AND AMARASINGHE, S. Tiramisu: A polyhedral compiler for expressing fast and portable code. In *2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)* (2019), IEEE, pp. 193–205.
- [4] BONDHUGULA, U., HARTONO, A., RAMANUJAM, J., AND SARDAYAPPAN, P. A practical automatic polyhedral parallelizer and locality optimizer. In *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation* (2008), pp. 101–113.
- [5] CHEN, H., FAN, B., COLLINS, A., HAGEDORN, B., GABUROV, E., MASUDA, M., BROOKHART, M., SULLIVAN, C., KNIGHT, J., ZHANG, Z., ET AL. Tawa: Automatic warp specialization for modern gpus with asynchronous references. *arXiv preprint arXiv:2510.14719* (2025).
- [6] CHEN, T., MOREAU, T., JIANG, Z., ZHENG, L., YAN, E., SHEN, H., COWAN, M., WANG, L., HU, Y., CEZE, L., ET AL. {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)* (2018), pp. 578–594.
- [7] CHENG, Y., WANG, L., SHI, Y., XIA, Y., MA, L., XUE, J., WANG, Y., MO, Z., CHEN, F., YANG, F., ET AL. Pipethreader: Software-defined pipelining for efficient dnn execution. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)* (2025).
- [8] CHRIS, L., AND TODD, W. Xla: Tensorflow, compiler!, 2017. TensorFlow Dev Summit.
- [9] CORPORATION, N. Cuda basic linear algebra subroutine library. <https://docs.nvidia.com/cuda/cublas/index.html>.
- [10] CORPORATION, N. Nvidia tesla v100 gpu architecture, 2017. Technical report.
- [11] CORPORATION, N. Nvidia tesla a100 gpu architecture, 2020. Technical report.
- [12] DAO, T. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691* (2023).
- [13] DAO, T., AND GU, A. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060* (2024).
- [14] FEAUTRIER, P. Some efficient solutions to the affine scheduling problem. I. One-dimensional time. *International Journal of Parallel Programming* 21, 5 (1992), 313–347.
- [15] GROSSER, T., GROESSLINGER, A., AND LENGAUER, C. Polly—performing polyhedral optimizations on a low-level intermediate representation. *Parallel Processing Letters* 22, 04 (2012), 1250010.
- [16] LATTNER, C., AMINI, M., BONDHUGULA, U., COHEN, A., DAVIS, A., PIENAAR, J., RIDDLE, R., SHPEISMAN, T., VASILACHE, N., AND ZINENKO, O. Mlir: Scaling compiler infrastructure for domain specific computation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)* (2021), IEEE, pp. 2–14.
- [17] LI, M., LIU, Y., LIU, X., SUN, Q., YOU, X., YANG, H., LUAN, Z., GAN, L., YANG, G., AND QIAN, D. The deep learning compiler: A comprehensive survey. *IEEE Transactions on Parallel and Distributed Systems* 32, 3 (2020), 708–727.
- [18] LINKEDIN. Liger-kernel: Efficient triton kernels for llm training. <https://github.com/linkedin/Liger-Kernel>, 2024.
- [19] LIU, S., QI, C., CAO, Y., YANG, C., HU, W., SHI, X., YANG, F., AND YANG, M. Uncovering nested data parallelism and data reuse in dnn computation with fractaltensor. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (2024), pp. 160–177.
- [20] NEWTON, R., SCHLIMBACH, F., HAMPTON, M., AND KNOBE, K. Capturing and composing parallel patterns with intel cnc. In *Proc. of USENIX Workshop on Hot Topics in Parallelism (HotPar 2010)*, Berkeley, CA, USA (2010).
- [21] NVIDIA CORPORATION. Nvidia h100 tensor core gpu architecture, 2022. Technical report.
- [22] NVIDIA CORPORATION. Cutlass, 2024.
- [23] PASZKE, A., GROSS, S., MASSA, F., LERER, A., BRADBURY, J., CHANAN, G., KILLEEN, T., LIN, Z., GIMELSHEIN, N., ANTIGA, L., ET AL. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
- [24] RAGAN-KELLEY, J., BARNES, C., ADAMS, A., PARIS, S., DURAND, F., AND AMARASINGHE, S. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation* (2013), pp. 519–530.
- [25] SBIRLEA, A., POUCHET, L.-N., AND SARKAR, V. Dfgr an intermediate graph representation for macro-dataflow programs. In *2014 Fourth Workshop on Data-Flow Execution Models for Extreme Scale Computing* (2014), IEEE, pp. 38–45.

- [26] SHI, Y., YANG, Z., XUE, J., MA, L., XIA, Y., MIAO, Z., GUO, Y., YANG, F., AND ZHOU, L. Welder: Scheduling deep learning memory access via tile-graph. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)* (2023), pp. 701–718.
- [27] SPECTOR, B. F., ARORA, S., SINGHAL, A., FU, D. Y., AND RÉ, C. Thunderkittens: Simple, fast, and adorable ai kernels. *arXiv preprint arXiv:2410.20399* (2024).
- [28] TILLET, P., KUNG, H.-T., AND COX, D. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages* (2019), pp. 10–19.
- [29] VASWANI, A. Attention is all you need. *Advances in Neural Information Processing Systems* (2017).
- [30] WANG, L., CHENG, Y., SHI, Y., TANG, Z., MO, Z., XIE, W., MA, L., XIA, Y., XUE, J., YANG, F., ET AL. Tilelang: A composable tiled programming model for ai systems. *arXiv preprint arXiv:2504.17577* (2025).
- [31] WANG, L., MA, L., CAO, S., ZHANG, Q., XUE, J., SHI, Y., ZHENG, N., MIAO, Z., YANG, F., CAO, T., ET AL. Ladder: Enabling efficient {Low-Precision} deep learning computing through hardware-aware tensor transformation. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)* (2024), pp. 307–323.
- [32] ZHAO, J., LI, B., NIE, W., GENG, Z., ZHANG, R., GAO, X., CHENG, B., WU, C., CHENG, Y., LI, Z., ET AL. Akg: automatic kernel generation for neural processing units using polyhedral transformations. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (2021), pp. 1233–1248.
- [33] ZHENG, L., JIA, C., SUN, M., WU, Z., YU, C. H., HAJ-ALI, A., WANG, Y., YANG, J., ZHUO, D., SEN, K., ET AL. Ansor: Generating {High-Performance} tensor programs for deep learning. In *14th USENIX symposium on operating systems design and implementation (OSDI 20)* (2020), pp. 863–879.
- [34] ZHENG, S., CHEN, R., WEI, A., JIN, Y., HAN, Q., LU, L., WU, B., LI, X., YAN, S., AND LIANG, Y. Amos: enabling automatic mapping for tensor computations on spatial accelerators with hardware abstraction. In *Proceedings of the 49th Annual International Symposium on Computer Architecture* (2022), pp. 874–887.
- [35] ZHENG, S., CHEN, S., SONG, P., CHEN, R., LI, X., YAN, S., LIN, D., LENG, J., AND LIANG, Y. Chimera: An analytical optimizing framework for effective compute-intensive operators fusion. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)* (2023), IEEE, pp. 1113–1126.
- [36] ZHU, H., WU, R., DIAO, Y., KE, S., LI, H., ZHANG, C., XUE, J., MA, L., XIA, Y., CUI, W., ET AL. {ROLLER}: Fast and efficient tensor compilation for deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)* (2022), pp. 233–248.